

Identity & Access Service

Endpoints — Identity & Access Service

Este servicio cubre autenticación (`/auth`) y gestión del perfil propio (`/profile`). La autenticación está delegada a **Supabase Auth** (`auth.users`); la tabla `profiles` es la extensión de datos del negocio vinculada por UUID. Los endpoints son wrappers sobre el SDK de Supabase más operaciones sobre `profiles` .

Nota sobre Supabase Auth

Supabase mantiene dos capas separadas:

Capa	Tabla	Qué contiene
Auth	<code>auth.users</code> (interna)	email, password hash, tokens de sesión
Negocio	<code>public.profiles</code>	name, cellphoneNumber, role, trustPoints, institutionId

Cualquier cambio de **email o contraseña** pasa por el SDK de Supabase (que actualiza `auth.users`). Los demás campos del perfil se actualizan directamente en `profiles` . Nunca se almacena la contraseña en `profiles` .

Autenticación

POST /auth/register

Crea una cuenta nueva. Internamente: (1) llama a `supabase.auth.signUp` para crear el registro en `auth.users` , (2) inserta una fila en `profiles` con el UUID retornado.

URL Params:
None

Headers:
Content-Type: application/json

Data Params:

```
{
  "name": "string",
  "email": "string",
  "password": "string",
  "cellphoneNumber": "string"
}
```

Success Response:

Code: 201

Content:

```
{
  "message": "Registro exitoso. Revisa tu correo para confirmar tu cuenta.",
  "user": {
    "id": "uuid",
    "name": "Juan Pérez",
    "email": "juan@mail.com",
    "cellphoneNumber": "+56912345678",
    "role": "usuario_base",
    "trustPoints": 0,
    "institution": null
  }
}
```

Supabase envía automáticamente un correo de confirmación. El usuario no puede iniciar sesión hasta confirmar.

Error Response:

Code: 400

Content: { "error": "Missing required fields" }

OR

Code: 409

Content: { "error": "Email already registered" }

OR

Code: 500

Content: { "error": "Internal server error" }

POST /auth/login

Autentica al usuario con email y contraseña. Internamente llama a `supabase.auth.signInWithPassword`.

URL Params:

None

Headers:

Content-Type: application/json

Data Params:

```
{
  "email": "string",
  "password": "string"
}
```

Success Response:

Code: 200

Content:

```
{
  "access_token": "string",
  "refresh_token": "string",
  "user": {
    "id": "uuid",
    "name": "Juan Pérez",
    "email": "juan@mail.com",
    "cellphoneNumber": "+56912345678",
    "role": "usuario_base",
    "trustPoints": 42,
    "institution": null
  }
}
```

El `access_token` debe incluirse en el header `Authorization: Bearer <token>` en todas las llamadas autenticadas. Expira según la configuración de Supabase (por defecto 1 hora). Usar `refresh_token` para renovarlo.

Error Response:

Code: 400

Content: `{ "error": "Missing email or password" }`

OR

Code: 401

Content: `{ "error": "Invalid credentials" }`

OR

Code: 403

Content: `{ "error": "Email not confirmed" }`

OR

Code: 500

Content: `{ "error": "Internal server error" }`

POST /auth/logout

Cierra la sesión activa del usuario. Internamente llama a `supabase.auth.signOut`, que invalida el `refresh_token` en el servidor.

URL Params:

None

Data Params:

None

Headers:

Authorization: Bearer `<Auth Token>`

Content-Type: application/json

Success Response:

Code: 204

Error Response:

Code: 401

Content: `{ "error": "You are unauthorized to make this request." }`

POST /auth/password-recovery

Paso 1 del flujo de recuperación. El usuario solicita el enlace de reseteo desde la pantalla "¿Olvidaste tu contraseña?".

Recibe el email del usuario y delega en `supabase.auth.resetPasswordForEmail` el envío del correo con el enlace de recuperación. El enlace redirige al usuario a la app con un token de recuperación en la URL.

URL Params:

None

Headers:

Content-Type: application/json

Data Params:

```
{
  "email": "string"
}
```

Success Response:

Code: 200

Content:

```
{
  "message": "Si el correo está registrado, recibirás un enlace de recuperación en los próximos minutos."
}
```

La respuesta es siempre la misma sin importar si el email existe o no, para evitar enumeración de usuarios.

Error Response:

Code: 400

Content: `{ "error": "Missing email" }`

OR

Code: 500

Content: `{ "error": "Internal server error" }`

POST /auth/new-password

Paso 2 del flujo de recuperación. El usuario llega a esta pantalla tras hacer clic en el enlace del correo. Supabase adjunta un `access_token` de tipo `recovery` en la URL (`#access_token=...&type=recovery`). El cliente lo captura, establece la sesión, y llama a este endpoint.

Actualiza la contraseña del usuario autenticado via token de recuperación. Internamente llama a `supabase.auth.updateUser({ password })`.

URL Params:

None

Headers:

Authorization: Bearer `<Recovery Token>`

Content-Type: application/json

Data Params:

```
{
  "password": "string",
  "passwordConfirm": "string"
}
```

Success Response:

Code: 200

Content:

```
{
  "message": "Contraseña actualizada correctamente."
}
```

Error Response:

Code: 400

Content: `{ "error": "Passwords do not match" }`

OR

Code: 400

Content: `{ "error": "Password must be at least 8 characters" }`

OR

Code: 401

Content: `{ "error": "Invalid or expired recovery token" }`

OR

Code: 500

Content: `{ "error": "Internal server error" }`

Flujo completo de recuperación de contraseña

1. POST /api/auth/recuperar-contrasena { email }

↓

Supabase envía correo con enlace mágico

2. Usuario hace clic → navegador abre la app con:
https://app.geojana.cl/auth/nueva-contrasena
#access_token=<token>&type=recovery

3. El cliente (PWA) captura el token del fragment de la URL
y lo usa como Bearer token

4. POST /api/auth/nueva-contrasena { password, passwordConfirm }

Authorization: Bearer <Recovery Token>

↓

Supabase actualiza auth.users

Respuesta: { message: "Contraseña actualizada correctamente." }

5. Redirigir al usuario al login

Profile

Objeto perfil:

```
{
  "id": "uuid",
  "name": "string",
  "email": "string",
  "cellphoneNumber": "string",
  "role": "usuario_base | institucion | admin",
  "trustPoints": "int",
  "institution": {
    "id": "int",
    "name": "string"
  }
}
```

`trustPoints` es de solo lectura — lo gestiona el sistema según validaciones de reportes. `role` e `institution` solo los puede modificar un `admin`. `institution` es `null` para `usuario_base`.

GET /profile

Obtiene el perfil completo del usuario autenticado, extraído desde el token.

URL Params:

None

Data Params:

None

Headers:

Authorization: Bearer <Auth Token>

Role: usuario_base, institucion, admin.

Content-Type: application/json

Success Response:

Code: 200

Content:

```
{
  "id": "a1b2c3d4-...",
  "name": "Juan Pérez",
  "email": "juan@mail.com",
  "cellphoneNumber": "+56912345678",
  "role": "usuario_base",
  "trustPoints": 42,
  "institution": null
}
```

Error Response:

Code: 401

Content: { "error": "You are unauthorized to make this request." }

PATCH /profile

Actualiza los campos generales del perfil: `name` y `cellphoneNumber` . Solo se modifican los campos enviados. Opera únicamente sobre la tabla `profiles` .

URL Params:

None

Data Params:

```
{
  "name": "string",
  "cellphoneNumber": "string"
}
```

No se aceptan `email` , `password` , `role` , `trustPoints` ni `institutionId` en este endpoint. Cada uno tiene su propio endpoint o flujo.

Headers:

Authorization: Bearer <Auth Token>

Role: usuario_base, institucion, admin.

Content-Type: application/json

Success Response:

Code: 200

Content:

```
{
  "id": "a1b2c3d4-...",
  "name": "Juan Pablo Pérez",
  "email": "juan@mail.com",
  "cellphoneNumber": "+56987654321",
  "role": "usuario_base",
  "trustPoints": 42,
  "institution": null
}
```

Error Response:

Code: 400

Content: { "error": "No valid fields to update" }

OR

Code: 401

Content: { "error": "You are unauthorized to make this request." }

PATCH /profile/email

Flujo de dos pasos. Cambiar el email requiere confirmación de Supabase — el usuario debe hacer clic en un enlace enviado al nuevo correo. Hasta entonces, el email sigue siendo el anterior.

Solicita el cambio de email del usuario autenticado. Internamente llama a `supabase.auth.updateUser({ email: newEmail })`. Supabase envía un correo de confirmación al nuevo email. Una vez confirmado, actualiza `auth.users` y un trigger/webhook actualiza `profiles.email` en sincronía.

URL Params:

None

Data Params:

```
{
  "newEmail": "string",
  "password": "string"
}
```

Se requiere la contraseña actual para confirmar la identidad antes de iniciar el cambio de email.

Headers:

Authorization: Bearer <Auth Token>

Role: usuario_base, institucion, admin.

Content-Type: application/json

Success Response:

Code: 200

Content:

```
{
  "message": "Se envió un correo de confirmación a juan_nuevo@mail.com. El cambio se aplicará al confirmar."
}
```

Error Response:

Code: 400

Content: { "error": "Missing required fields" }

OR

Code: 401

Content: { "error": "Invalid password" }

OR

Code: 409

Content: { "error": "Email already in use" }

OR

Code: 500

Content: { "error": "Internal server error" }

PATCH /profile/password

Este endpoint es para usuarios **ya autenticados** que quieren cambiar su contraseña desde la configuración de cuenta. Es distinto al flujo de recuperación (`/auth/nueva-contrasena`), que opera con un token de recovery sin sesión activa.

Cambia la contraseña del usuario autenticado. Internamente verifica la contraseña actual contra Supabase (`signInWithPassword`) y luego llama a `supabase.auth.updateUser({ password: newPassword })` .

URL Params:

None

Data Params:

```
{
  "currentPassword": "string",
  "newPassword": "string",
  "newPasswordConfirm": "string"
}
```

Headers:

Authorization: Bearer `<Auth Token>`

Role: usuario_base, institucion, admin.

Content-Type: application/json

Success Response:

Code: 200

Content:

```
{  
  "message": "Contraseña actualizada correctamente."  
}
```

Error Response:

Code: 400

Content: { "error": "Passwords do not match" }

OR

Code: 400

Content: { "error": "New password must be at least 8 characters" }

OR

Code: 401

Content: { "error": "Current password is incorrect" }

OR

Code: 500

Content: { "error": "Internal server error" }